האוניברסיטה העברית
בי"ס להנדסה ולמדעי המחשב

<u>בחינה בקורסי מבוא למדעי המחשב:</u>
67101 – מבוא למדעי המחשב
67130 – מבוא למדעי המחשב בדגש על תכנות פרוצדוראלי

מועד **ב'** - סמסטר א' - תשס"ז – 2006-2007
המורה: פרופ' ג'ף רוזנשיין
משך הבחינה: שלוש שעות

ת.ז. _____
מס' מחברת: _____
תאריך: 25.05.2007

<u>הנחיות:</u>

1. יש לענות על 3 מתוך 4 השאלות הבאות (3 בלבד). משקל כל השאלות זהה (33 נק'). במידה ויוגשו פתרונות ליותר משלוש שאלות, הציון יקבע על פי שלוש השאלות הראשונות בלבד.
2. **יש לכתוב פתרון לכל אחת מהשאלות שבחרת <u>במחברת נפרדת</u>. יש לציין על כריכת המחברת לאיזה שאלה המחברת מתייחסת, באופן קריא וברור.**
3. **במידה ובחרתם לענות על השאלות האמריקאיות עליכם לסמן את תשובתכם על גבי טופס המבחן עצמו ולהגיש אותו יחד עם מחברות הבחינה.**
4. יש לציין את מס' תעודת הזהות ומספר המחברת על **כל** מחברת וגם **על גבי טופס הבחינה עצמו**.
5. הקפידו הקפדה יתרה על סדר וכתב יד קריא. בחינה בלתי קריאה לא תזכה את כותבה במלוא הנקודות.
6. יש למחוק טיוטות באופן ברור.
7. **יש להשאיר שוליים.**
8. **אסור להשתמש בעט אדום.**
9. הקפידו על סגנון תכנותי נאות: כתבו באופן מודולארי, אל תכפילו קוד וכו'. מותר להוסיף פונקציות עזר מעבר לפונקציות הנדרשות בשאלה.
10. תעדו כל תכנית כך שניתן יהיה להבינה בנקל (מותר לתעד בעברית).
11. פתרון מיטבי לשאלה יחשב פתרון **פשוט ויעיל ככל האפשר**. תוכנית מסורבלת או מסובכת לא תתקבל כתשובה מלאה (גם אם היא נכונה).
12. **אין להקצות** או להשתמש במבני נתונים פרט לאלו שהוגדרו בכל שאלה. עם זאת, מותר תמיד להשתמש במבני נתונים נוספים אשר גודלם ומספרם אינו תלוי בקלט (למשל מותר להקצות מערך בן תא אחד פעם אחת).
13. בכל השאלות מותר להניח שהקלט חוקי (ומקיים את תנאי השאלה).

שאלה 1: שאלות אמריקאיות (33 נק')

ענו על **8 מתוך 10** הסעיפים הבאים בלבד. סמנו באופן ברור את שני הסעיפים עליהם בחרתם לא לענות.
עבור כל שאלה, בחרו את התשובה הנכונה ביותר. ערכו של כל סעיף 4 נקודות (נק' אחת מקבלים חינם).
סמנו בבירור את תשובותיכם **על גבי טופס המבחן** והגישו אותו.
ודאו כי רשמתם את מס' תעודת הזהות שלכם ואת מספר המחברת בראש טופס הבחינה!

א. מהו החסם העליון ההדוק ביותר על זמן הריצה של foo(n)? (ניתן להניח כי 0=<n):

a. O(n)
b. O(log n)
c. $O(n^2)$
d. O(n*log(n))

```
static void foo(int n) {
    if (n<=1) {
        return;
    }
    int i=1;
    while (i<=n) {
        i=i+2;
    }
    foo(n/2);
}
```

ב. תהיינה C,B,A מחלקות. בהנתן קטע קוד משמאל.
מה מהבאים אפשרי? (בחר בטענה הנכונה ביותר)

```
...
A a = new B();
C c = (C) a;
...
```

a. C יורשת מ B שיורשת מ A.
b. B יורשת מ C שיורשת מ A.
c. B יורשת מ A שיורשת מ C.
d. תשובות ב+ג נכונות.
e. תשובות א+ג נכונות.

ג. תהי L רשימה משורשרת חד כיוונית בעלת n איברים, נניח שיש ברשותנו מצביע לראש הרשימה וmצביע לזנבה (לאיבר האחרון). איזה מהפעולות הבאות תהיה הכי **פחות** יעילה מבחינה אסימפטוטית?

a. מחיקת איבר מראש הרשימה.
b. מחיקת איבר מזנב הרשימה.
c. הוספת איבר לראש הרשימה.
d. הוספת איבר לזנב הרשימה.
e. כל הפעולות יעילות במידה שווה.

ד. נניח שנתון הקוד משמאל. איזה מהבאים הוא כותרת (prototype) חוקית לשיטה במחלקה היורשת מהמחלקה Base?

```
class Base {
    public int foo(int i){
        ...
    }
}
```

1. public double foo(int i)
2. public int foo(int s)
3. private int foo(int i)
4. private int foo(double i)
5. private void foo(double i)

a. השני בלבד
b. השני, השלישי והרביעי
c. השני, הרביעי והחמישי
d. הראשון והרביעי
e. כל הכותרות חוקיות

ה. מה יהיה הפלט של התוכנית הבאה (ההדפסות משמאל לימין):

a. 3 3 7 3
b. 3 7 3 3
c. 7 3 7 3
d. 3 7 3 7

```
public class Main {
    public static void main(String[] args){
        int [] arr= {3,7};
        System.out.print(arr[0]+" ");
        g(arr);
        System.out.print(arr[1]+" ");
    }

    public static void g(int[] arr){
        arr[0] = arr[0]+arr[1];
        arr[1] = arr[0]-arr[1];
        arr[0] = arr[0]-arr[1];
        System.out.print(arr[0]+" ");
        System.out.print(arr[1]+" ");
    }
}
```

ו. מהו זמן הריצה של חיפוש ריבועי (quadratic search) במקרה הטוב ביותר?

a. $O(n^2)$
b. $O(n)$
c. $O(1)$
d. $O(\log n)$

ז. תהי A מחלקה אבסטרקטית (abstract class), איזה מהטענות הבאות נכונה?

a. כל שיטה במחלקה A חייבת להיות מוגדרת כשיטה אבסטרקטית (abstract method) בעצמה.
b. למחלקה A אין בנאי (constructor).
c. כל מחלקה היורשת מ A חייבת **בכל מקרה** לממש את כל השיטות האבסטרקטיות של A.
d. יכול להיות שבמחלקה A אין אף שיטה אבסטרקטית.
e. בחר בסעיף זה אם כל הטענות אינן נכונות.

ח. נתונה המחלקה C (משמאל).
איזו טענה מבין הטענות הבאות נכונה?

a. התוכנית לא תעבור קומפילציה כי f אינה פונקציה סטטית.
b. התוכנית לא תעבור קומפילציה כי g היא פונקציה private.
c. התוכנית לא תעבור קומפילציה כי ה main חייב להיות המתודה הסטטית היחידה במחלקה.
d. התוכנית תעבור קומפילציה אך תתרסק בזמן ריצה כי f אינה סטטית.

```
class C {
    public void f(){...}
    private static void g(){...}
    static void h(){...}
    public static void main(String [] args){
        f(x);  // [1]
        g(x);  // [2]
        h(x);  // [3]
    }
```

ט. מה תחזיר השיטה foo(321,123)?

a. 222
b. 444
c. 124
d. 321
e. כל התשובות שגויות.

```
int foo(int a, int b) {
    if (a==0) {
        return b;
    }
    return foo(a-1,b+1);
}
```

י. איזו מבין הטענות הבאות נכונה?

a. אפשר לממש ממשק (interface) שיטה המוגדרת בממשק אחר.
b. כל משתנה המוגדר בממשק הוא תמיד פומבי קבוע וסטטי (public, static, final).
c. ממשק יכול לרשת (להרחיב) גם מחלקות וגם ממשקים אחרים.
d. המילה השמורה implements מציינת, בין היתר, שממשק אחד יורש ממשק אחר.

שאלה 2 – Merge Sort (33 נק')

אלגוריתם המיון Merge Sort ממומש לעיתים קרובות על רשימות מקושרות, אולם ניתן לממשו גם עבור מערכים. אחד השלבים החשובים במימוש האלגוריתם במערכים הוא מימוש שיטה שמסוגלת למזג שני מקטעים ממויינים שמופיעים זה לצד זה במערך.

לדוגמא, בציור מתואר מערך הנקרא data שבו שני אזורים ממויינים: הראשון מתחיל במיקום data[2] ועד data[5] והשני מתא data[6] ועד data[7]

32	95	16	23	24	66	15	19	2	54

לאחר מיזוג שני המקטעים, נוצר מקטע ממוין רציף אחד מתא data[2] ועד תא data[7] המכיל את הערכים משני המקטעים ששולבו לפי הסדר. שאר המערך (בתאים data[0] עד data[1] ובתאים data[8] עד data[9]) נותר ללא כל שינוי.

32	95	15	16	19	23	24	66	2	54

סעיף א' (11 נק'):

כיתבו שיטה לא רקורסיבית

public void merge(int[ ] data, int lo, int mid, int hi)

הממזגת שני מקטעים ממויינים צמודים, הראשון בטווח מ data[lo] ועד data[mid] והשני בטווח data[mid+1] ועד data[hi] (שימו לב שהערך של mid אינו בהכרח שווה ל (lo+hi)/2 אלא יכול להיות כל ערך ביניים). הניחו כי כל אחד משני המקטעים כבר ממוין בסדר עולה בזמן הקריאה לשיטה. לאחר סיום הריצה, כל האזור מ data[lo] ועד data[hi] צריך להיות ממוין בסדר עולה.

במהלך הפתרון אסור להשתמש במבני נתונים נוספים המוקצים ב- heap, אך מותר להשתמש במשתנים מקומיים המוגדרים בתוך שיטת merge() עצמה.

סעיף ב' (2 נק'):

מהי סיבוכיות זמן הריצה של השיטה merge() שמימשתם בסעיף א' במקרה הגרוע ביותר? הסבירו את תשובתכם.

סעיף ג' (11 נק'):

כתבו שיטה לא רקורסיבית

public void merge2(int[ ] data, int lo, int mid, int hi)

הפותרת את אותה הבעייה המופיעה בסעיף א'. הפעם מותר (ואף מומלץ) להקצות אובייקטים חדשים ב-heap. עליכם לוודא שסיבוכיות זמן הריצה במקרה הגרוע ביותר של שיטה זו תהיה טובה יותר מזו של השיטה שמימשתם בסעיף א'.

סעיף ד' (2 נק'):

מהי סיבוכיות זמן הריצה במקרה הגרוע ביותר של השיטה **merge2()** שמימשתם בסעיף ג'? הסבירו את תשובתכם.

סעיף ה' (4 נק'):

כתבו את השיטה הרקורסיבית הבאה:

public void mergeSort(int[ ] data, int lo, int hi)

הממיינת בסדר עולה את התאים בטווח lo עד hi (כולל) בתוך מערך בשם data. בעת מימוש השיטה מותר לקרוא למתודה merge() מסעיף א' או למתודה merge2() מסעיף ב' גם אם לא עניתם על אחד מהסעיפים הללו.

סעיף ו' (3 נק'):

[הקפידו לענות על שני חלקי הסעיף]. הניחו שבעת מימוש **mergeSort()** השתמשתם בשיטה merge() מסעיף א'.
מהי סיבוכיות זמן הריצה במקרה הגרוע ביותר במימוש זה? הסבירו.
כעת, הניחו שהשתמשתם דוקא בשיטה **merge2()** מסעיף ג'. מהי סיבוכיות זמן הריצה במקרה הגרוע ביותר במימוש זה? הסבירו.

שאלה 3 – פתרון מבוך (33 נק')

נייצג מבוך על ידי מערך דו-מימדי של מספרים שלמים (int), כאשר תא המכיל את הערך 0 מייצג תא פנוי, תא המכיל את הערך 1- מייצג קיר, ותא המכיל את הערך 2- מייצג את תא המטרה. תנועה במבוך אפשרית רק בין שני תאים שכנים לאורך שורה או עמודה (ללא אלכסונים). התנועה היא חוקית רק אם שני התאים השכנים ריקים (מכילים את הערך 0), או אם התא אליו אנו זזים הוא תא המטרה (מכיל את הערך 2-).
מבוך חוקי יוגדר כמערך דו-מימדי שגודלו גדול מאפס, כל תאיו מכילים אחד מהערכים המוזכרים לעיל (0, 1-, או 2-) ויש בו בדיוק תא מטרה יחיד.

עליכם לממש פונקציה סטטית השייכת למחלקה ללא data members, שטיפוסה:
public static boolean solveMaze(int[] maze, int row, int col)
אשר פותרת את המבוך באופן רקורסיבי.
הפרמטרים לפונקציה הם המבוך (maze) ומיקום תא המוצא (row,col).

אם המבוך אינו פתיר (אין מסלול אפשרי מתא המוצא לתא המטרה) הפונקציה תחזיר ערך false ותאי המערך לא ישונו (אם שונו במהלך הריצה עליכם לדאוג שיחזרו לערכיהם ההתחלתיים).
אם קיים מסלול מתא המוצא לתא המטרה הפונקציה תחזיר ערך true והמסלול שמצאתם יסומן על המערך הדו מימדי באופן הבא: תא המוצא יכיל את הערך 1, התא הבא במסלול יכיל את הערך 2, וכן הלאה עד התא האחרון במסלול אשר צמוד לתא המטרה (הערך בתא המטרה יישאר 2-). המסלול שמצאתם אינו חייב להיות המסלול הקצר ביותר.

לדוגמא:

המבוך (ותא המוצא 0,5):

	0	1	2	3	4	5	6	7
0	0	0	0	0	0	**0**	0	0
1	0	-1	-1	-1	-1	-1	-1	-1
2	0	-1	0	0	0	0	0	-1
3	0	-1	0	**-2**	0	-1	0	-1
4	0	-1	0	0	0	-1	0	-1
5	0	-1	-1	-1	-1	-1	0	-1
6	0	-1	0	0	0	0	0	-1
7	0	-1	-1	0	0	-1	-1	-1
8	0	0	0	0	0	0	0	0

פתרון אפשרי:

	0	1	2	3	4	5	6	7
0	***6***	***5***	***4***	***3***	***2***	***1***	0	0
1	***7***	-1	-1	-1	-1	-1	-1	-1
2	***8***	-1	0	0	***28***	***27***	***26***	-1
3	***9***	-1	0	**-2**	***29***	-1	***25***	-1
4	***10***	-1	0	***31***	***30***	-1	***24***	-1
5	***11***	-1	-1	-1	-1	-1	***23***	-1
6	***12***	-1	0	***19***	***20***	***21***	***22***	-1
7	***13***	-1	-1	***18***	0	-1	-1	-1
8	***14***	***15***	***16***	***17***	0	0	0	0

הנחיות:

- הינכם יכולים להניח שקלט הפונקציה חוקי (מערך הקלט מכיל מבוך התחלתי חוקי והתא ההתחלתי ריק).
- אינכם צריכים להגדיר קבועים עבור הערכים המייצגים את סוגי התאים השונים אלא יכולים להשתמש ישירות בערכים אלו בקוד (מספרי קסם).
- הינכם יכולים להגדיר פונקציות עזר נוספות במחלקה, ובלבד שיהיו סטטיות. כמו כן, הינכם יכולים להגדיר קבועים סטטיים במחלקה.
- המבוך יכול להכיל מעגלים, ותא המטרה אינו חייב להיות בצמוד לקיר. לכן, שיטת החיפוש הפשוטה בה מתקדמים כאשר הקיר תמיד צמוד לימין (או לשמאל) לא תעבוד, ו***לא תתקבל כפתרון.***
- לא ניתן לנוע מחוץ לגבולות המערך הדו-מימדי.

שאלה 4 -- מציאת מקטעים דומים ברשימות. (33 נק')

לרשותכם שתי מחלקות, Node ו- Stack אותן כבר מימשנו עבורכם ואין צורך שתממשו אותן בעצמכם.

המחלקה Node מייצגת איבר (node) ברשימה מקושרת (linked list) המחזיקה מספרים מטיפוס int והיא בעלת שתי השיטות הבאות:

```
public Node getNext()
```
שיטה זו מחזירה את ה- Node הבא ברשימה או null אם אין עוד nodes ברשימה.

```
public int getData()
```
שיטה זו מחזירה את המספר המוכל ב- node הנוכחי.

המחלקה Stack מייצגת מחסנית של מספרים מסוג int והיא בעלת 4 השיטות הבאות:

```
public Stack()
```
שיטה זו יוצרת מחסנית חדשה ריקה מאיברים.

```
public int pop()
```
שיטה זו מסירה את האיבר בראש המחסנית ומחזירה אותו. במידה והמחסנית ריקה, קריאה לשיטה זו תגרום לשגיאה ועליכם להימנע ממצב זה.

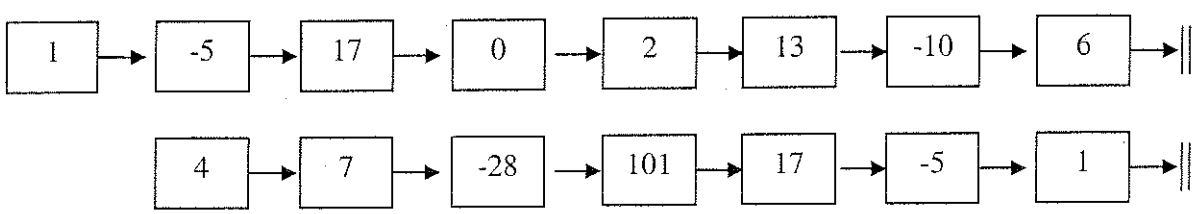
```
public void push(int number)
```
שיטה זו מכניסה את המספר number לראש המחסנית ודוחפת את האיברים המקוריים במחסנית מקום אחד מטה.

```
public boolean isEmpty()
```
שיטה זו מחזירה ערך בוליאני של אמת אם ורק אם המחסנית ריקה (אינה מכילה איברים).
הניחו כי כל פעולות המחסנית עובדות בסיבוכיות זמן של $O(1)$.

סעיף א' (13 נק'):

כתבו שיטה (לא רקורסיבית, סטטית, המוגדרת במחלקה ללא data members) בשם `findSegment()`. שיטה זו מקבלת שני פרמטרים מטיפוס Node המייצגים שתי רשימות מקושרות, ומדפיסה את סדרת המספרים הארוכה ביותר המהווה רישא (prefix) של הרשימה הראשונה וסיפא (suffix) הפוכה של הרשימה השנייה (ראה דוגמא).
כל מספר יודפס בשורה חדשה לפי סדר הופעתו ברשימה הראשונה.

```
public static void findSegment(Node first, Node second)
```
לדוגמא, עבור הקלט הבא



שיטה זו תדפיס:
```
1
-5
17
```

עליכם להניח כי שני הפרמטרים המועברים לשיטה זו אינם null והם מייצגים רשימות מקושרות חוקיות (ללא מעגלים, ללא nodes משותפים, המסתיימות ב-null).

שימו לב:
i. אין להקצות זיכרון נוסף על ה-heap מעבר לאובייקט מחסנית אחד שאתם רשאים לעשות בו שימוש.
ii. אין לשנות את מבנה הרשימות המקושרות או את הערכים המוכלים בהן.

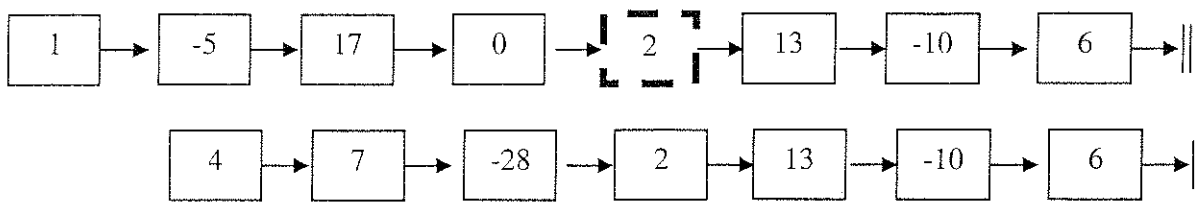
על השיטה לעבוד בסיבוכיות זמן של $O(n)$. הראו כי עמדתם בדרישה זו.
מותר לכתוב שיטות נוספות במידת הצורך ולהיעזר בהן בפתרון השאלה.

סעיף ב' (20 נק'):

כתבו שיטה (סטטית, המוגדרת במחלקה ללא data members) בשם `findMaxSuffix()`.

```
public static Node findMaxSuffix(Node first, Node second)
```

שיטה זו מקבלת שני פרמטרים מטיפוס Node המייצגים שתי רשימות מקושרות ומחזירה מצביע לקודקוד ברשימה הראשונה (first) המהווה את תחילת הסיפא הזהה הארוכה ביותר בשתי הרשימות. במידה ולרשימות אין סיפא זהה, שיטה זו צריכה להחזיר null.
לדוגמא, עבור הקלט הבא:



השיטה `findMaxSuffix()` תחזיר מצביע לקודקוד שמוקף בקו מרוסק.

עליכם להניח כי שני הפרמטרים המועברים לשיטה זו אינם null והם מייצגים רשימות מקושרות חוקיות (ללא מעגלים, המסתיימות ב-null).

שימו לב:

i. שתי הרשימות אינן מכילות בהכרח אותו מספר של nodes, כלומר אין להסתמך על כך שאורכן זהה.
ii. אנו עוסקים בסיפא זהה מבחינת ערכים אך לא משותפת. כלומר אין nodes המשותפים לשתי הרשימות.
iii. מותר לכם להשתמש ברקורסיה אך ניתן ואף מומלץ שלא להשתמש ברקורסיה.
iv. אין להקצות בשיטה זו זיכרון נוסף על ה-heap.

על השיטה `findMaxSuffix()` לעבוד בסיבוכיות זמן של $O(n)$. הראו כי עמדתם בדרישה זו.